

3. PROJECT DESIGN PHASE

3.1 System Architecture

The application is designed using a layered architecture consisting of a presentation layer, application-logic layer and data layer. The presentation layer contains pages, forms, tables, filters and charts. The application layer validates user input, applies cricket-statistic rules and coordinates workflows. The data layer stores and retrieves player, team, match, innings and performance information.

A typical workflow begins when a user selects an operation, such as registering a player or recording a match. The interface collects values and performs basic checks. The application layer verifies business rules, including whether the selected player and match exist and whether the values are valid. The data layer saves the record, preferably using a transaction for related updates. When a report is requested, the application retrieves filtered records, calculates metrics and returns results for display.

For a small academic prototype, Python with a lightweight interface framework and SQLite may be sufficient. A larger deployment could use a server database and authentication service. Regardless of technology, database queries and statistical formulas should not be embedded directly into visual components. Separating responsibilities makes calculation functions easier to test and interface changes easier to maintain.

The architecture should also define how errors are handled. Validation errors should be shown to users in plain language. Unexpected failures should be logged for troubleshooting without exposing sensitive information or internal database details. Configuration and secrets should be kept separate from application source code.

3.2 Database and Entity Relationship Design

The core database entities are Player, Team, Match, Innings, BattingPerformance, BowlingPerformance and FieldingPerformance. Player stores identity and playing attributes. Team stores team information. Match stores date, venue, format and participating teams.

Innings connects a match to the batting and bowling teams. Performance tables connect players to innings and store scorecard values.

Primary keys uniquely identify records, while foreign keys maintain relationships. A player can have multiple performance records, and a match can contain multiple innings. Unique constraints can prevent duplicate performance entries for the same player and innings. If players change teams, historical records should preserve the team context relevant to the match rather than automatically rewriting previous results.

Canonical input values should be stored, while statistics derived can be calculated when requested. Storing every calculated value may create inconsistency if source values are later corrected. If calculated values are cached for performance, the system needs a reliable refresh strategy. Created and updated timestamps can support traceability and help administrators identify recent changes.

Deletion should be handled carefully. Permanently deleting a player who appears in historical matches can break reports or relationships. Deactivation or soft deletion may be safer. The database design should be reviewed against common workflows and tested with sample records before the interface is built around it.

3.3 User Interface and Navigation Design

The interface should reflect the user's most common tasks. A dashboard can provide recent matches, registered player counts and quick navigation. A player page should show profile information, match history and selected season or career statistics. A match page should display participating teams, innings and scorecard entry forms. A reports page should provide filters and export options.

Forms should group related fields and use familiar cricket terminology. Required fields should be marked clearly. Numeric fields should indicate accepted values and units. Bowling overs should be entered using a consistent method, such as total legal balls or separate overs and balls, and displayed in conventional notation where appropriate. Validation messages should explain how to correct an entry without forcing the user to re-enter unrelated fields.

Charts should answer specific questions. A runs-per-match chart should label dates or match sequence on the horizontal axis and runs on the vertical axis. Bowling economy and

wickets use different units and should not be combined on a misleading shared scale. Each chart should include a title, units, selected filters and an indication of the number of matches included.

Tables should support readable sorting and clear empty states. Color should not be the only way to communicate status. The interface should be tested at the screen size expected for the project and should maintain readable text and sensible spacing. Consistency across pages reduces the time users need to learn the application.

3.4 Security, Validation and Error Handling Design

User-entered values should be treated as untrusted until validated. Required fields, numeric ranges, dates and relationships must be checked before saving. The system should distinguish zero from missing values and should reject impossible or inconsistent records. Validation should occur in the application logic even if the interface also provides immediate feedback.

Related database updates should use transactions, so a failure does not leave partial records. Errors shown to users should be understandable and should not expose passwords, private information, stack traces or raw database messages. If authentication is implemented, passwords must be securely hashed, and access permissions must be checked for every restricted operation.

The system should minimize collection of personal information and restrict access to exports. Reports should include only fields necessary for their purpose. If multiple users share the application, roles such as administrator, scorer, coach and read-only player can be defined according to actual needs. Hiding a button is not sufficient security; permission must be enforced by application logic.

Backup and recovery should be planned. A local prototype can document how to copy the database safely. A deployed system should define backup frequency, retention and restoration tests. These safeguards protect data integrity and make the application more dependable for ongoing use.